

# **MODULE 2**

## FUNCTIONAL TESTING

# BOUNDARY VALUE ANALYSIS

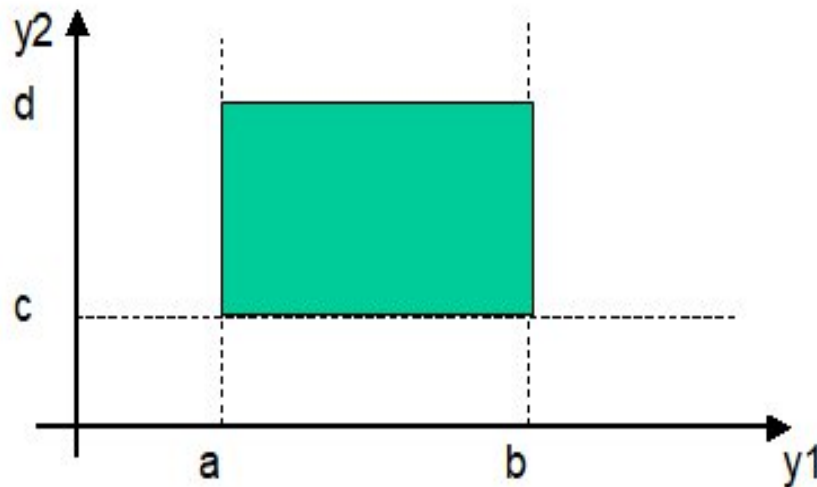
- BVA – Black box testing technique, where implementation is not known.
- Boundary testing is the process of testing between extreme ends or boundaries between partitions of the input values.
- Boundary value analysis focuses on the boundary of the input space to identify test cases.
- The rationale behind boundary value analysis is that errors tend to occur near the extreme values of an input variable.
- In our discussion we will assume a program  $P$  accepting two inputs  $y_1$  and  $y_2$  such that  $a \leq y_1 \leq b$  and  $c \leq y_2 \leq d$

## Valid Input for Program P

- consider the following function:

$$f(y_1, y_2), \text{ where } a \leq y_1 \leq b, c \leq y_2 \leq d$$

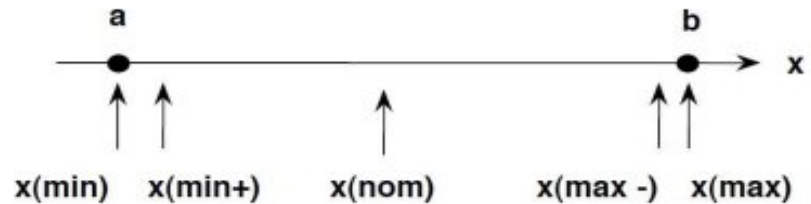
- boundary inequalities of  $n$  input variables define an  $n$ -dimensional input space:



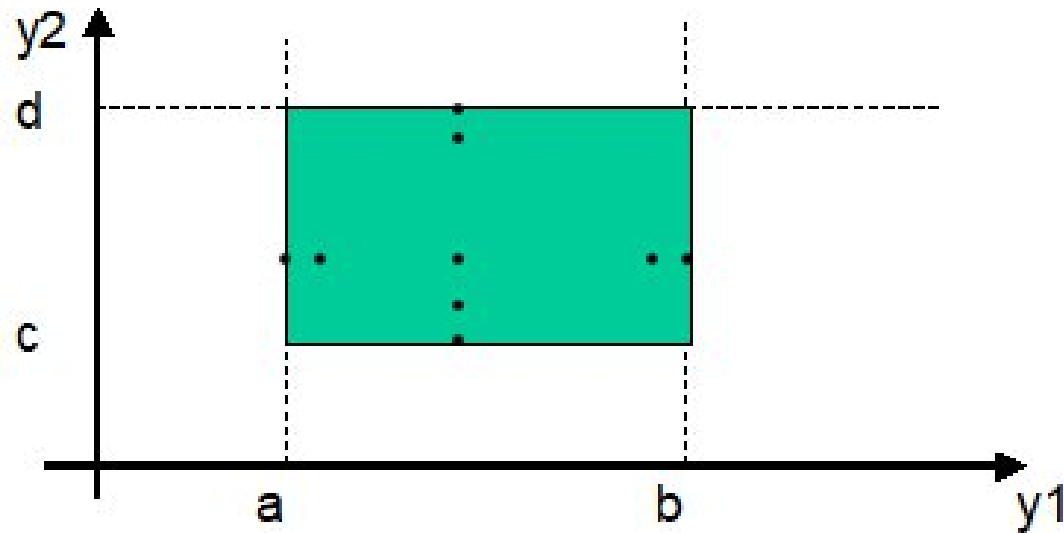
# Value Selection in Boundary Value Analysis

The basic idea in boundary value analysis is to select input variable values at their:

- Minimum
- Just above the minimum
- A nominal value
- Just below the maximum
- Maximum



# BOUNDARY VALUE ANALYSIS FOR PROGRAM P



$$T = \{ \langle y_{1nom}, y_{2min} \rangle, \langle y_{1nom}, y_{2min+} \rangle, \langle y_{1nom}, y_{2nom} \rangle, \langle y_{1nom}, y_{2max-} \rangle, \\ \langle y_{1nom}, y_{2max+} \rangle, \langle y_{1min}, y_{2nom} \rangle, \langle y_{1min+}, y_{2nom} \rangle, \langle y_{1max-}, y_{2nom} \rangle, \\ \langle y_{1max}, y_{2nom} \rangle \}$$

# Single Fault Assumption

- Boundary value analysis is also augmented by the single fault assumption principle.

“Failures occur rarely as the result of the simultaneous occurrence of two (or more) faults”

## Generalizing Boundary Value Analysis

- The basic boundary value analysis can be generalized in two ways:
- By the number of variables -  $(4n + 1)$  test cases for  $n$  variables
- By the kinds of ranges of variables
- Programming language dependent
- Bounded discrete
- Unbounded discrete (no upper or lower bounds clearly defined)
- Logical variables

## **Limitations of Boundary Value Analysis**

- BVA tests only the values at, just below, and just above the boundaries. It may miss defects that occur with values well inside the valid or invalid input ranges.
- BVA is mainly applicable to numeric inputs or ordered data types. It's not effective for testing unordered data.
- Boundary value testing cannot find errors related to functional logic, system behavior, performance issues.
- BVA requires clearly defined boundary values in the requirements. If boundaries are ambiguous, incomplete, or missing, the test coverage will be inadequate.



## Robustness Testing

Robustness testing is a simple extension of boundary value analysis. In addition to the five boundary value analysis values of variables, we add values slightly greater than the maximum (max+) and a value slightly less than the minimum (min-).

The main value of robustness testing is to force attention on exception handling.

In some strongly typed languages values beyond the predefined range will cause a run-time error.

It is a choice of using a weak typed language with exception handling or a strongly typed language with explicit logic to handle out of range values.

Number of test cases =  $6n + 1$  for  $n$  inputs

# Robustness Test Case for program P

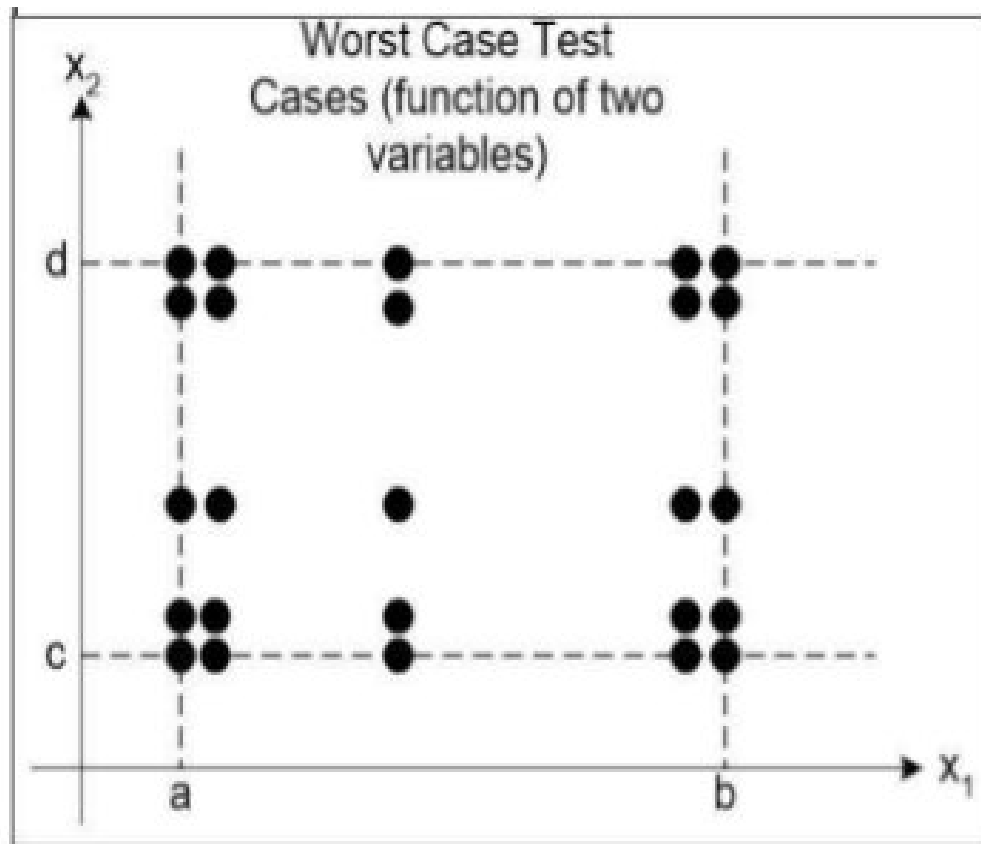


|                                       |
|---------------------------------------|
| $\langle X_{1nom}, X_{2min} \rangle$  |
| $\langle X_{1nom}, X_{2min+} \rangle$ |
| $\langle X_{1nom}, X_{2nom} \rangle$  |
| $\langle X_{1nom}, X_{2max-} \rangle$ |
| $\langle X_{1nom}, X_{2max} \rangle$  |
| $\langle X_{1min}, X_{2nom} \rangle$  |
| $\langle X_{1min+}, X_{2nom} \rangle$ |
| $\langle X_{1nom}, X_{2nom} \rangle$  |
| $\langle X_{1max}, X_{2nom} \rangle$  |
| $\langle X_{1max}, X_{2nom} \rangle$  |
| $\langle X_{1min}, X_{2nom} \rangle$  |
| $\langle X_{1nom}, X_{2max+} \rangle$ |
| $\langle X_{1max+}, X_{2nom} \rangle$ |
| $\langle X_{1nom}, X_{2min-} \rangle$ |

# Worst Case Testing

- **Worst case testing** is a method used to check how a system behaves under the most extreme conditions of its input variables.
- In worst case testing we reject the single fault assumption which means we are interested what happens when more than one variable has an extreme value.
- Considering that we have five different values that can be considered during boundary value analysis testing for one variable, now we take the Cartesian product of these possible values for 2, 3, ... n variables.
- In this respect we can have  **$5^n$  test cases** for n input variables.
- Worst case testing can be further augmented by considering **robust worst case testing** (i.e. adding slightly out of bounds values to the five already considered).

# Worst Case Testing for Program P



$\langle x_{1min}, x_{2min} \rangle$   
 $\langle x_{1min}, x_{2min+} \rangle$   
 $\langle x_{1min}, x_{2nom} \rangle$   
 $\langle x_{1min}, x_{2max-} \rangle$   
 $\langle x_{1min}, x_{2max} \rangle$   
 $\langle x_{1min+}, x_{2min} \rangle$   
 $\langle x_{1min+}, x_{2min+} \rangle$   
 $\langle x_{1min+}, x_{2nom} \rangle$   
 $\langle x_{1min+}, x_{2max-} \rangle$   
 $\langle x_{1min+}, x_{2max} \rangle$   
 $\langle x_{1nom}, x_{2min} \rangle$   
 $\langle x_{1nom}, x_{2min+} \rangle$   
 $\langle x_{1nom}, x_{2nom} \rangle$   
 $\langle x_{1nom}, x_{2max-} \rangle$   
 $\langle x_{1nom}, x_{2max} \rangle$

$\langle x_{1max}, x_{2min} \rangle$   
 $\langle x_{1max}, x_{2min+} \rangle$   
 $\langle x_{1max}, x_{2nom} \rangle$   
 $\langle x_{1max}, x_{2max-} \rangle$   
 $\langle x_{1max}, x_{2max} \rangle$   
 $\langle x_{1max}, x_{2min} \rangle$   
 $\langle x_{1max}, x_{2min+} \rangle$   
 $\langle x_{1max}, x_{2nom} \rangle$   
 $\langle x_{1max}, x_{2max-} \rangle$   
 $\langle x_{1max}, x_{2max} \rangle$

# Test Cases for the Triangle Problem

**Table 5.1 Normal Boundary Value Test Cases**

| <i>Case</i> | <i>a</i> | <i>b</i> | <i>c</i> | <i>Expected Output</i> |
|-------------|----------|----------|----------|------------------------|
| 1           | 100      | 100      | 1        | Isosceles              |
| 2           | 100      | 100      | 2        | Isosceles              |
| 3           | 100      | 100      | 100      | Equilateral            |
| 4           | 100      | 100      | 199      | Isosceles              |
| 5           | 100      | 100      | 200      | Not a triangle         |
| 6           | 100      | 1        | 100      | Isosceles              |
| 7           | 100      | 2        | 100      | Isosceles              |
| 8           | 100      | 100      | 100      | Equilateral            |
| 9           | 100      | 199      | 100      | Isosceles              |
| 10          | 100      | 200      | 100      | Not a triangle         |
| 11          | 1        | 100      | 100      | Isosceles              |
| 12          | 2        | 100      | 100      | Isosceles              |
| 13          | 100      | 100      | 100      | Equilateral            |
| 14          | 199      | 100      | 100      | Isosceles              |
| 15          | 200      | 100      | 100      | Not a triangle         |

**Table 5.2 (Selected) Worst-Case Boundary Value Test Cases**

| <i>Case</i> | <i>a</i> | <i>b</i> | <i>c</i> | <i>Expected Output</i> |
|-------------|----------|----------|----------|------------------------|
| 1           | 1        | 1        | 1        | Equilateral            |
| 2           | 1        | 1        | 2        | Not a triangle         |
| 3           | 1        | 1        | 100      | Not a triangle         |
| 4           | 1        | 1        | 199      | Not a triangle         |
| 5           | 1        | 1        | 200      | Not a triangle         |
| 6           | 1        | 2        | 1        | Not a triangle         |
| 7           | 1        | 2        | 2        | Isosceles              |
| 8           | 1        | 2        | 100      | Not a triangle         |
| 9           | 1        | 2        | 199      | Not a triangle         |
| 10          | 1        | 2        | 200      | Not a triangle         |
| 11          | 1        | 100      | 1        | Not a triangle         |
| 12          | 1        | 100      | 2        | Not a triangle         |
| 13          | 1        | 100      | 100      | Isosceles              |
| 14          | 1        | 100      | 199      | Not a triangle         |
| 15          | 1        | 100      | 200      | Not a triangle         |
| 16          | 1        | 199      | 1        | Not a triangle         |
| 17          | 1        | 199      | 2        | Not a triangle         |
| 18          | 1        | 199      | 100      | Not a triangle         |
| 19          | 1        | 199      | 199      | Isosceles              |
| 20          | 1        | 199      | 200      | Not a triangle         |
| 21          | 1        | 200      | 1        | Not a triangle         |
| 22          | 1        | 200      | 2        | Not a triangle         |
| 23          | 1        | 200      | 100      | Not a triangle         |
| 24          | 1        | 200      | 199      | Not a triangle         |
| 25          | 1        | 200      | 200      | Isosceles              |

{0,1,2,100,199,200,201}

Key Differences:

**Number of Variables at Boundary:**

- Normal BVA: Tests one variable at its boundary while others remain typical.
- Worst-Case BVA: Tests all variables at their boundaries in different combinations.

**Number of Test Cases:**

- Normal BVA results in fewer test cases since you test each variable individually.
- Worst-Case BVA results in more test cases since you're testing the cross-product of all boundary values.

# The NextDate problem

- The NextDate problem is a function of three variables: day, month and year. Upon the input of
- a certain date it returns the date of the day after that of the input. The input variables have the
- obvious conditions:
- $1 \leq \text{Day} \leq 31$ .
- $1 \leq \text{month} \leq 12$ .
- $1812 \leq \text{Year} \leq 2012$ .



- In worst-case testing, we generate all combinations of the boundary values. This leads to 5 test values for each of the variables (lower boundary, upper boundary, just outside boundaries), resulting in  $5 \times 5 \times 5 = 125$  test cases.
- Worst-case testing generates a comprehensive set of test cases by covering every possible boundary scenario for the variables—ensuring that the NextDate function is tested against all edge cases and extreme input conditions.

**Examples of Worst-Case Test Values:**

Day = 1, Month = 1, Year = 1900 (lowest boundary for all inputs)

Day = 31, Month = 12, Year = 2100 (highest boundary for all inputs)

Day = 0, Month = 6, Year = 1950 (invalid day, normal month, and mid-range year)

Day = 15, Month = 0, Year = 1950 (valid day, invalid month)

Day = 29, Month = 2, Year = 2020 (valid leap year case)

Table 5.3 Worst-Case Test Cases

| Case | Month | Day | Year | Expected Output |
|------|-------|-----|------|-----------------|
| 1    | 1     | 1   | 1812 | 1, 2, 1812      |
| 2    | 1     | 1   | 1813 | 1, 2, 1813      |
| 3    | 1     | 1   | 1912 | 1, 2, 1912      |
| 4    | 1     | 1   | 2011 | 1, 2, 2011      |
| 5    | 1     | 1   | 2012 | 1, 2, 2012      |
| 6    | 1     | 2   | 1812 | 1, 3, 1812      |
| 7    | 1     | 2   | 1813 | 1, 3, 1813      |
| 8    | 1     | 2   | 1912 | 1, 3, 1912      |
| 9    | 1     | 2   | 2011 | 1, 3, 2011      |
| 10   | 1     | 2   | 2012 | 1, 3, 2012      |
| 11   | 1     | 15  | 1812 | 1, 16, 1812     |
| 12   | 1     | 15  | 1813 | 1, 16, 1813     |
| 13   | 1     | 15  | 1912 | 1, 16, 1912     |
| 14   | 1     | 15  | 2011 | 1, 16, 2011     |
| 15   | 1     | 15  | 2012 | 1, 16, 2012     |
| 16   | 1     | 30  | 1812 | 1, 31, 1812     |
| 17   | 1     | 30  | 1813 | 1, 31, 1813     |
| 18   | 1     | 30  | 1912 | 1, 31, 1912     |
| 19   | 1     | 30  | 2011 | 1, 31, 2011     |
| 20   | 1     | 30  | 2012 | 1, 31, 2012     |
| 21   | 1     | 31  | 1812 | 2, 1, 1812      |
| 22   | 1     | 31  | 1813 | 2, 1, 1813      |
| 23   | 1     | 31  | 1912 | 2, 1, 1912      |
| 24   | 1     | 31  | 2011 | 2, 1, 2011      |
| 25   | 1     | 31  | 2012 | 2, 1, 2012      |
| 26   | 2     | 1   | 1812 | 2, 2, 1812      |
| 27   | 2     | 1   | 1813 | 2, 2, 1813      |
| 28   | 2     | 1   | 1912 | 2, 2, 1912      |

(continued)

**Table 5.3 Worst-Case Test Cases (Continued)**

| Case | Month | Day | Year | Expected Output |
|------|-------|-----|------|-----------------|
| 29   | 2     | 1   | 2011 | 2, 2, 2011      |
| 30   | 2     | 1   | 2012 | 2, 2, 2012      |
| 31   | 2     | 2   | 1812 | 2, 3, 1812      |
| 32   | 2     | 2   | 1813 | 2, 3, 1813      |
| 33   | 2     | 2   | 1912 | 2, 3, 1912      |
| 34   | 2     | 2   | 2011 | 2, 3, 2011      |
| 35   | 2     | 2   | 2012 | 2, 3, 2012      |
| 36   | 2     | 15  | 1812 | 2, 16, 1812     |
| 37   | 2     | 15  | 1813 | 2, 16, 1813     |
| 38   | 2     | 15  | 1912 | 2, 16, 1912     |
| 39   | 2     | 15  | 2011 | 2, 16, 2011     |
| 40   | 2     | 15  | 2012 | 2, 16, 2012     |
| 41   | 2     | 30  | 1812 | Invalid date    |
| 42   | 2     | 30  | 1813 | Invalid date    |
| 43   | 2     | 30  | 1912 | Invalid date    |
| 44   | 2     | 30  | 2011 | Invalid date    |
| 45   | 2     | 30  | 2012 | Invalid date    |
| 46   | 2     | 31  | 1812 | Invalid date    |
| 47   | 2     | 31  | 1813 | Invalid date    |
| 48   | 2     | 31  | 1912 | Invalid date    |
| 49   | 2     | 31  | 2011 | Invalid date    |
| 50   | 2     | 31  | 2012 | Invalid date    |
| 51   | 6     | 1   | 1812 | 6, 2, 1812      |
| 52   | 6     | 1   | 1813 | 6, 2, 1813      |
| 53   | 6     | 1   | 1912 | 6, 2, 1912      |
| 54   | 6     | 1   | 2011 | 6, 2, 2011      |
| 55   | 6     | 1   | 2012 | 6, 2, 2012      |
| 56   | 6     | 2   | 1812 | 6, 3, 1812      |
| 57   | 6     | 2   | 1813 | 6, 3, 1813      |

(continued)

**Table 5.3 Worst-Case Test Cases (Continued)**

| <i>Case</i> | <i>Month</i> | <i>Day</i> | <i>Year</i> | <i>Expected Output</i> |
|-------------|--------------|------------|-------------|------------------------|
| 58          | 6            | 2          | 1912        | 6, 3, 1912             |
| 59          | 6            | 2          | 2011        | 6, 3, 2011             |
| 60          | 6            | 2          | 2012        | 6, 3, 2012             |
| 61          | 6            | 15         | 1812        | 6, 16, 1812            |
| 62          | 6            | 15         | 1813        | 6, 16, 1813            |
| 63          | 6            | 15         | 1912        | 6, 16, 1912            |
| 64          | 6            | 15         | 2011        | 6, 16, 2011            |
| 65          | 6            | 15         | 2012        | 6, 16, 2012            |
| 66          | 6            | 30         | 1812        | 7, 1, 1812             |
| 67          | 6            | 30         | 1813        | 7, 1, 1813             |
| 68          | 6            | 30         | 1912        | 7, 1, 1912             |
| 69          | 6            | 30         | 2011        | 7, 1, 2011             |
| 70          | 6            | 30         | 2012        | 7, 1, 2012             |
| 71          | 6            | 31         | 1812        | Invalid date           |
| 72          | 6            | 31         | 1813        | Invalid date           |
| 73          | 6            | 31         | 1912        | Invalid date           |
| 74          | 6            | 31         | 2011        | Invalid date           |
| 75          | 6            | 31         | 2012        | Invalid date           |
| 76          | 11           | 1          | 1812        | 11, 2, 1812            |
| 77          | 11           | 1          | 1813        | 11, 2, 1813            |
| 78          | 11           | 1          | 1912        | 11, 2, 1912            |
| 79          | 11           | 1          | 2011        | 11, 2, 2011            |
| 80          | 11           | 1          | 2012        | 11, 2, 2012            |
| 81          | 11           | 2          | 1812        | 11, 3, 1812            |
| 82          | 11           | 2          | 1813        | 11, 3, 1813            |
| 83          | 11           | 2          | 1912        | 11, 3, 1912            |
| 84          | 11           | 2          | 2011        | 11, 3, 2011            |
| 85          | 11           | 2          | 2012        | 11, 3, 2012            |
| 86          | 11           | 15         | 1812        | 11, 16, 1812           |

(continued)

**Table 5.3 Worst-Case Test Cases (Continued)**

| <i>Case</i> | <i>Month</i> | <i>Day</i> | <i>Year</i> | <i>Expected Output</i> |
|-------------|--------------|------------|-------------|------------------------|
| 87          | 11           | 15         | 1813        | 11, 16, 1813           |
| 88          | 11           | 15         | 1912        | 11, 16, 1912           |
| 89          | 11           | 15         | 2011        | 11, 16, 2011           |
| 90          | 11           | 15         | 2012        | 11, 16, 2012           |
| 91          | 11           | 30         | 1812        | 12, 1, 1812            |
| 92          | 11           | 30         | 1813        | 12, 1, 1813            |
| 93          | 11           | 30         | 1912        | 12, 1, 1912            |
| 94          | 11           | 30         | 2011        | 12, 1, 2011            |
| 95          | 11           | 30         | 2012        | 12, 1, 2012            |
| 96          | 11           | 31         | 1812        | Invalid date           |
| 97          | 11           | 31         | 1813        | Invalid date           |
| 98          | 11           | 31         | 1912        | Invalid date           |
| 99          | 11           | 31         | 2011        | Invalid date           |
| 100         | 11           | 31         | 2012        | Invalid date           |
| 101         | 12           | 1          | 1812        | 12, 2, 1812            |
| 102         | 12           | 1          | 1813        | 12, 2, 1813            |
| 103         | 12           | 1          | 1912        | 12, 2, 1912            |
| 104         | 12           | 1          | 2011        | 12, 2, 2011            |
| 105         | 12           | 1          | 2012        | 12, 2, 2012            |
| 106         | 12           | 2          | 1812        | 12, 3, 1812            |
| 107         | 12           | 2          | 1813        | 12, 3, 1813            |
| 108         | 12           | 2          | 1912        | 12, 3, 1912            |
| 109         | 12           | 2          | 2011        | 12, 3, 2011            |
| 110         | 12           | 2          | 2012        | 12, 3, 2012            |
| 111         | 12           | 15         | 1812        | 12, 16, 1812           |
| 112         | 12           | 15         | 1813        | 12, 16, 1813           |
| 113         | 12           | 15         | 1912        | 12, 16, 1912           |
| 114         | 12           | 15         | 2011        | 12, 16, 2011           |
| 115         | 12           | 15         | 2012        | 12, 16, 2012           |

(continued)

**Table 5.3 Worst-Case Test Cases (Continued)**

| Case | Month | Day | Year | Expected Output |
|------|-------|-----|------|-----------------|
| 116  | 12    | 30  | 1812 | 12, 31, 1812    |
| 117  | 12    | 30  | 1813 | 12, 31, 1813    |
| 118  | 12    | 30  | 1912 | 12, 31, 1912    |
| 119  | 12    | 30  | 2011 | 12, 31, 2011    |
| 120  | 12    | 30  | 2012 | 12, 31, 2012    |
| 121  | 12    | 31  | 1812 | 1, 1, 1813      |
| 122  | 12    | 31  | 1813 | 1, 1, 1814      |
| 123  | 12    | 31  | 1912 | 1, 1, 1913      |
| 124  | 12    | 31  | 2011 | 1, 1, 2012      |
| 125  | 12    | 31  | 2012 | 1, 1, 2013      |

# The Commission Problem

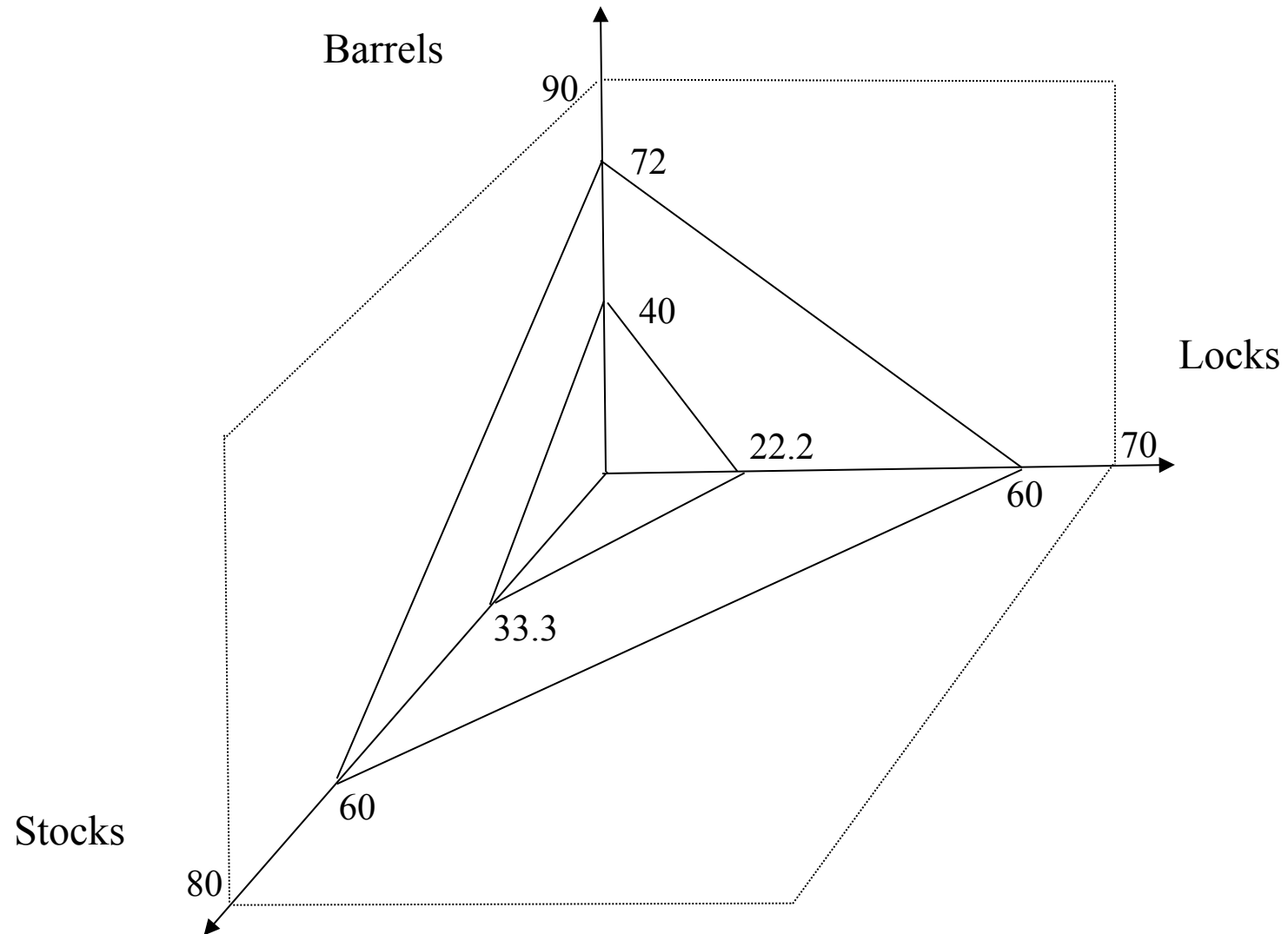
- Rifle salespersons in the Arizona Territory sold rifle locks, stocks, and barrels made by a gunsmith in Missouri
- Lock = \$45.00, stock = \$30.00, barrel = \$25.00
- Each salesperson had to sell at least one complete rifle per month (\$100)
- The most one salesperson could sell in a month was 70 locks, 80 stocks, and 90 barrels
- Each salesperson sent a telegram to the Missouri company with the total order for each town (s)he visits
- $1 \leq \text{towns visited} \leq 10$ , per month
- Commission: 10% on sales up to \$1000, 15% on the next \$800, and 20% on any sales in excess of \$1800

### **What You Should Test:**

- Test near \$1000: Try selling different combinations of locks, stocks, and barrels that result in a commission just below \$1000, exactly \$1000, and just above \$1000. This helps ensure the system calculates commissions correctly when crossing that threshold.
- Test near \$1800: Similarly, try combinations that give a commission just below, exactly, and just above \$1800. This checks that the commission percentage changes are handled properly when you reach \$1800 in sales.
- By testing around these threshold points, you can make sure the commission calculation works as expected at the key points where the percentage changes, without needing to test every single possible combination of locks, stocks, and barrels.



# Example Test Cases Using Output Range Values



# Output Boundary Value Test Cases

| Case # | Locks     | Stocks    | Barrels   | Sales       | Comm.      | Comments      |
|--------|-----------|-----------|-----------|-------------|------------|---------------|
| 1      | 1         | 1         | 1         | 100         | 10         | min           |
| 2      | 10        | 10        | 9         | 975         | 97.5       | border-       |
| 3      | 10        | 9         | 10        | 970         | 97         | border-       |
| 4      | 9         | 10        | 10        | 955         | 95.5       | border-       |
| 5      | <b>10</b> | <b>10</b> | <b>10</b> | <b>1000</b> | <b>100</b> | <b>border</b> |
| 6      | 10        | 10        | 11        | 1025        | 103.75     | border+       |
| 7      | 10        | 11        | 10        | 1030        | 104.5      | border+       |
| 8      | 11        | 10        | 10        | 1045        | 106.75     | border+       |

- The goal is to find combinations of input values (locks, stocks, barrels) that stress the system right around the threshold points (e.g., slightly below or above \$1000, \$1800).

| Aspect     | Boundary Value Testing                                                                       | Special Value Testing                                                                                           |
|------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Definition | Focuses on testing at the edges of input ranges, particularly at minimum and maximum values. | Focuses on specific values of interest based on the logic of the application.                                   |
| Goal       | To identify defects at the limits of input ranges where errors are likely to occur.          | To validate the correctness of functionality based on significant points, especially where logic changes occur. |

# Output Special Value Test Cases

| Case # | Locks | Stocks | Barrels | Sales       | Comm.  | Comment |
|--------|-------|--------|---------|-------------|--------|---------|
| 1      | 10    | 11     | 9       | <b>1005</b> | 100.75 | border+ |
| 2      | 18    | 17     | 19      | <b>1795</b> | 219.25 | Border- |
| 3      | 18    | 19     | 17      | <b>1805</b> | 221    | border+ |

# Equivalence Class Testing

# Equivalence Classes Testing

- Decreases the number of possible test cases.
- Equivalence class partitioning/Testing divides input data into similar equivalence classes.
- Divided sets are called equivalence classes or partitions.
- Then we pick only one value from each partitions for testing.
- The idea of equivalence class testing is to identify test cases by using one element from each equivalence class.

# Equivalence classes

- Equivalence Class Testing, which is also known as Equivalence Class Partitioning (ECP) and Equivalence Partitioning, is an important software testing technique used by the team of testers for grouping and partitioning of the test input data, which is then used for the purpose of testing the software product into a number of different classes.
- The idea of equivalence class testing is to identify test cases by using one element from each equivalence class.

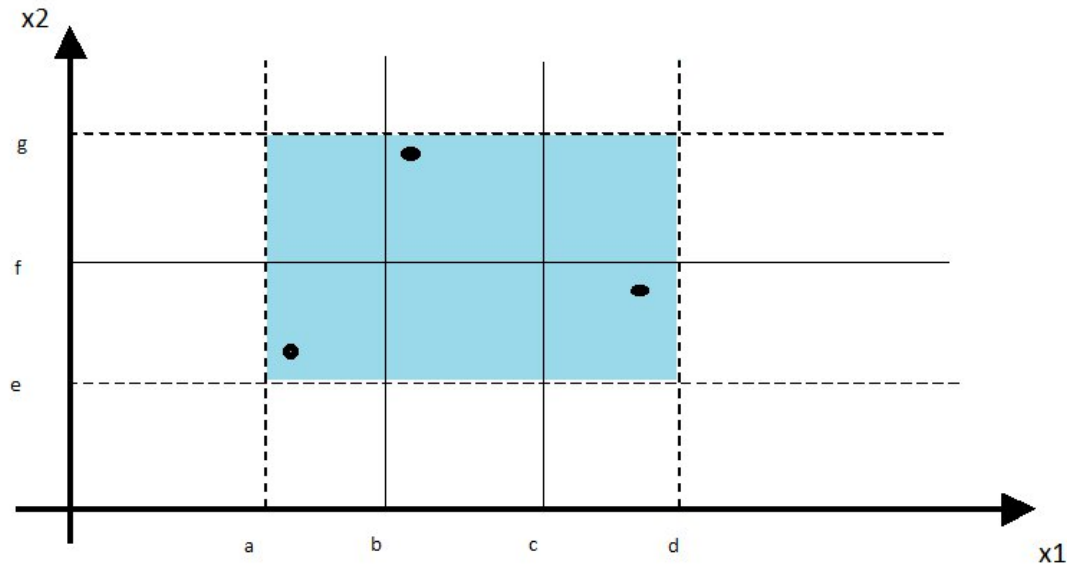
# Types of Equivalence testing

- There are four types of equivalence testing:
  - Weak Normal equivalence class testing
  - Strong Normal equivalence class testing
  - Weak Robust equivalence class testing
  - Strong Robust equivalence class testing



# Weak Normal Equivalence Class Testing

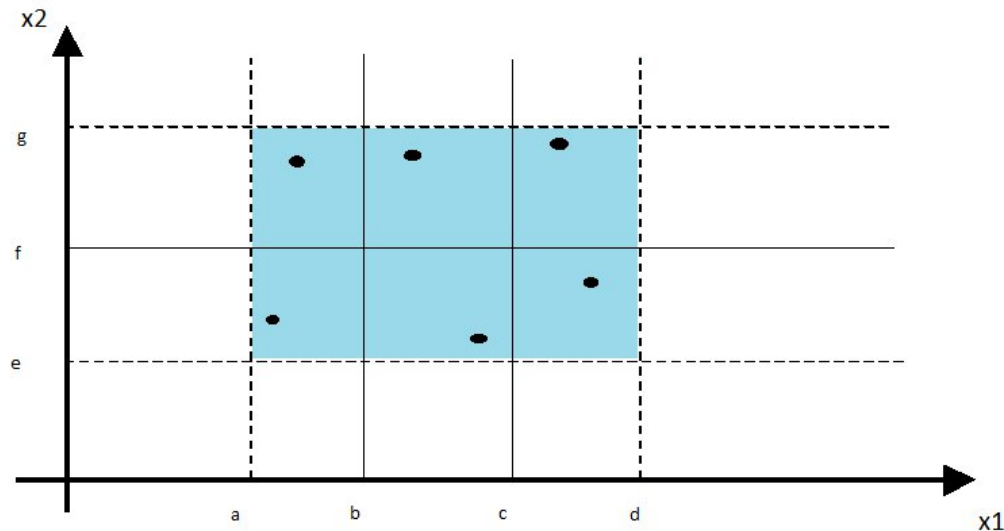
- The word weak means single fault assumption.
- Weak equivalence class testing is accomplished by using one variable from each equivalence class in a test case.
- The minimum number of test cases is equal to the number of classes in the partition with the largest number of subsets.



Each dot in above graph indicates a test data. From each class we have one dot meaning that there is one representative element of each test case. In fact, we will have, always, the same number of weak equivalence class test cases as the classes in the partition.

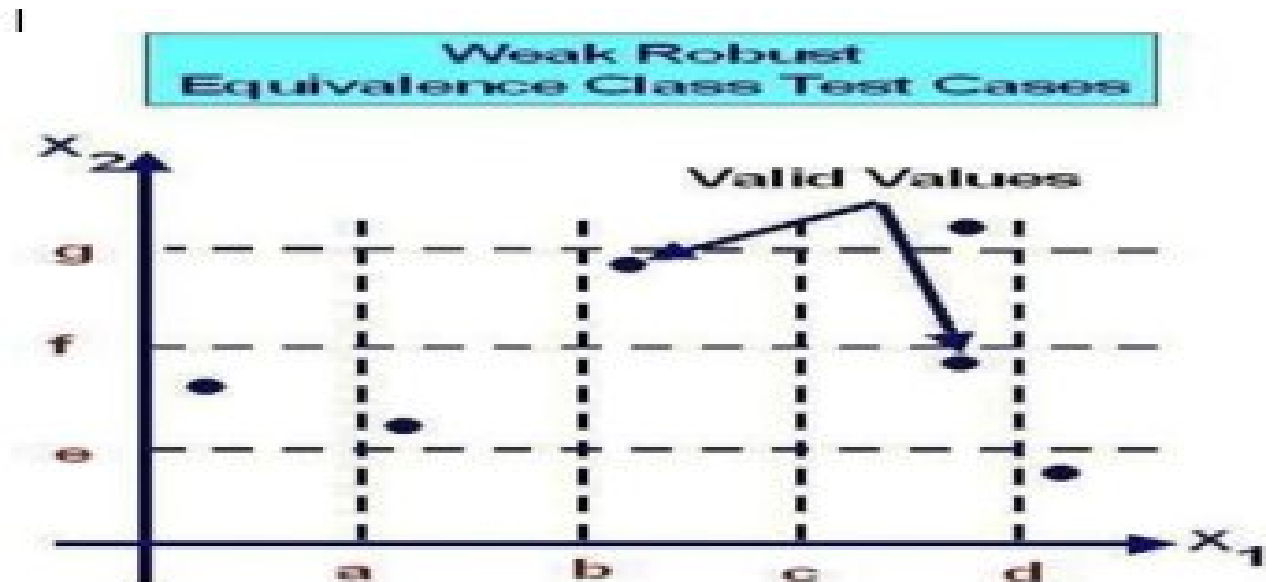
# Strong Normal Equivalence Class Testing

- Strong equivalence class testing is based on the Cartesian Product of the partition subsets (**multiple fault assumption**).
- Generates more test cases which test for any interaction between the representative values from each of the subsets.



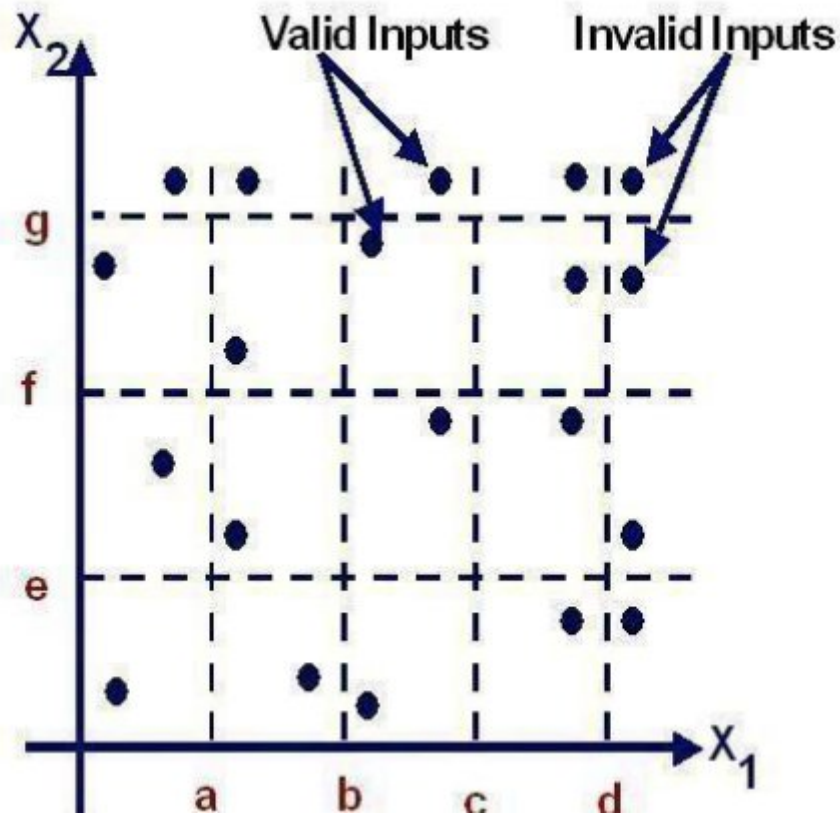
# Weak Robust Equivalence Class Testing

- Valid Inputs
  - For valid inputs, use one value from each valid class (as in what we have called weak equivalence class testing). In this context, each input in these test cases will be valid.
- Invalid Inputs
  - For invalid inputs, a test case will have one invalid value and the remaining values will be valid. In this context, a “single failure” should cause the test case to fail.



# Strong Robust Equivalence Class Testing

- Robust- from invalid values
- Strong- from redundant values



# Equivalence Test Cases for the Triangle Problem

- In the problem statement we note that there are four possible outputs:
  - Not a Triangle
  - Isosceles
  - Equilateral
  - Scalene
- We can use these to identify output (range) equivalence classes:

$R1 = \{ \langle a, b, c \rangle \mid \text{the triangle with sides } a, b, c, \text{ is equilateral} \}$

$R2 = \{ \langle a, b, c \rangle \mid \text{the triangle with sides } a, b, c, \text{ is isosceles} \}$

$R3 = \{ \langle a, b, c \rangle \mid \text{the triangle with sides } a, b, c, \text{ is scalene} \}$

$R4 = \{ \langle a, b, c \rangle \mid \text{sides } a, b, c \text{ do not form a triangle} \}$

- These classes yield the following simple set of test cases:

# Sample Test Cases based on Output Domain

| Test Case | a | b | c | Expected Output |
|-----------|---|---|---|-----------------|
| WN1       | 5 | 5 | 5 | Equilateral     |
| WN2       | 2 | 2 | 3 | Isosceles       |
| WN3       | 3 | 4 | 5 | Scalene         |
| WN4       | 4 | 1 | 2 | Not a Triangle  |

Table 1: Weak and Strong Normal class test cases

| <i>a</i> | <i>b</i> | <i>c</i> | <i>Expected Output</i>                             |
|----------|----------|----------|----------------------------------------------------|
| -1       | 5        | 5        | Value of a is not in the range of permitted values |
| 5        | -1       | 5        | Value of b is not in the range of permitted values |
| 5        | 5        | -1       | Value of c is not in the range of permitted values |
| 201      | 5        | 5        | Value of a is not in the range of permitted values |
| 5        | 201      | 5        | Value of b is not in the range of permitted values |
| 5        | 5        | 201      | Value of c is not in the range of permitted values |

Table 2: Weak Robust class test cases

| <i>Test Case</i> | <i>a</i> | <i>b</i> | <i>c</i> | <i>Expected Output</i>                                     |
|------------------|----------|----------|----------|------------------------------------------------------------|
| SR1              | -1       | 5        | 5        | Value of a is not in the range of permitted values         |
| SR2              | 5        | -1       | 5        | Value of b is not in the range of permitted values         |
| SR3              | 5        | 5        | -1       | Value of c is not in the range of permitted values         |
| SR4              | -1       | -1       | 5        | Values of a, b are not in the range of permitted values    |
| SR5              | 5        | -1       | -1       | Values of b, c are not in the range of permitted values    |
| SR6              | -1       | 5        | -1       | Values of a, c are not in the range of permitted values    |
| SR7              | -1       | -1       | -1       | Values of a, b, c are not in the range of permitted values |

Table 3: Strong Robust class test cases



# Equivalence Test Cases for the Triangle Problem (Input Domain)

- If we base the equivalence classes on the input domain, we will obtain a larger set of test cases. We can define the sets:

$$D1 = \{ \langle a, b, c \rangle \mid a=b=c \}$$

$$D2 = \{ \langle a, b, c \rangle \mid a=b, a \neq c \}$$

$$D3 = \{ \langle a, b, c \rangle \mid a=c, a \neq b \}$$

$$D4 = \{ \langle a, b, c \rangle \mid b=c, a \neq b \}$$

$$D5 = \{ \langle a, b, c \rangle \mid a \neq b, a \neq c, b \neq c \}$$

- As a separate property we can apply the triangle property to see even if the input constitutes a triangle

$$D6 = \{ \langle a, b, c \rangle \mid a \geq b+c \}$$

$$D7 = \{ \langle a, b, c \rangle \mid b \geq a+c \}$$

$$D8 = \{ \langle a, b, c \rangle \mid c \geq a+b \}$$

- If we wanted also we could split D6 into

$$D6' = \{ \langle a, b, c \rangle \mid a = b+c \} \text{ and}$$

$$D6'' = \{ \langle a, b, c \rangle \mid a > b+c \}$$

# Equivalence Test Cases for the NextDate Problem (Input Domain)

- Nextdate is a function of three variables, month, day, and year and these have ranges defined as:

$$\begin{aligned}1 &\leq \text{month} \leq 12 \\1 &\leq \text{day} \leq 31 \\1812 &\leq \text{year} \leq 2012\end{aligned}$$

- We will examine below the valid, invalid equivalence classes, strong, and weak equivalence class testing.

# Traditional Test Cases

- The valid equivalence classes are:

M1= {month |  $1 \leq \text{month} \leq 12$ }

D1= {day |  $1 \leq \text{day} \leq 31$ }

Y1= {year |  $1812 \leq \text{year} \leq 2012$ }

The invalid equivalence classes are:

M2= {month | month < 1}

M3= {month | month > 12}

D2= {day | day < 1}

D3= {day | day > 31}

Y2= {year | year < 1812}

Y3= {year | year > 2012}

These classes yield the following test cases, where the valid inputs are mechanically selected from the approximate middle of the valid range:

# Traditional Test Cases

| Case ID | Month | Day | Year | Expected Output |
|---------|-------|-----|------|-----------------|
| TE1     | 6     | 15  | 1912 | 6/16/1912       |
| TE2     | -1    | 15  | 1912 | Invalid         |
| TE3     | 13    | 15  | 1912 | Invalid         |
| TE4     | 6     | -1  | 1912 | Invalid         |
| TE5     | 6     | 32  | 1912 | Invalid         |
| TE6     | 6     | 15  | 1811 | Invalid         |
| TE7     | 6     | 15  | 2013 | Invalid         |

## **Summary of Test Case Strategy**

### **Weak Normal and Strong Normal Test Cases:**

- Focus on validating the basic functionality of the NextDate function without testing for invalid inputs.
- Example: Testing a date like 6/15/2000 to ensure the next date is computed correctly.

### **Weak Robust and Strong Robust Test Cases:**

- Include both valid and invalid inputs, ensuring the function handles erroneous data gracefully.
- Example: Testing for a negative day or month should produce an appropriate error message.

# Choice of Equivalence Classes

- If we more carefully chose the equivalence relation, the resulting equivalence classes will be more useful

M1= {month | month has 30 days}

M2= {month | month has 31 days}

M3= {month | month is February}

D1= {day |  $1 \leq \text{day} \leq 28$ }

D2= {day | day = 29}

D3= {day | day = 30}

D4= {day | day=31}

Y1= {year | year = 1900}

Y2= {year |  $1812 \leq \text{year} \leq 2012$  AND  $\text{year} \neq 1900$  AND  $(0 = \text{year mod } 4)$ }

Y3= {year |  $1812 \leq \text{year} \leq 2012$  AND  $0 \neq \text{year mod } 4$ }

# Strong Equivalence Test Cases

| CASE ID | Month | Day | Year | Output    |
|---------|-------|-----|------|-----------|
| SE1     | 6     | 14  | 1900 | 6/15/1900 |
| SE2     | 6     | 14  | 1912 | 6/15/1912 |
| SE3     | 6     | 14  | 1913 | 6/15/1913 |
| SE4     | 6     | 29  | 1900 | 6/30/1900 |
| SE5     | 6     | 29  | 1912 | 6/30/1912 |
| SE6     | 6     | 29  | 1913 | 6/30/1913 |
| SE7     | 6     | 30  | 1900 | 7/1/1900  |
| SE8     | 6     | 30  | 1912 | 7/1/1912  |
| SE9     | 6     | 30  | 1913 | 7/1/1913  |
| SE10    | 6     | 31  | 1900 | ERROR     |
| SE11    | 6     | 31  | 1912 | ERROR     |
| SE12    | 6     | 31  | 1913 | ERROR     |
| SE13    | 7     | 14  | 1900 | 7/15/1900 |
| SE14    | 7     | 14  | 1912 | 7/15/1912 |
| SE15    | 7     | 14  | 1913 | 7/15/1913 |
| SE16    | 7     | 29  | 1900 | 7/30/1900 |
| SE17    | 7     | 29  | 1912 | 7/30/1912 |
| SE18    | 7     | 29  | 1913 | 7/30/1913 |

# Strong Equivalence Test Classes

| CASE ID | Month | Day | Year | Output    |
|---------|-------|-----|------|-----------|
| SE19    | 7     | 30  | 1900 | 7/31/1900 |
| SE20    | 7     | 30  | 1912 | 7/31/1912 |
| SE21    | 7     | 30  | 1913 | 7/31/1913 |
| SE22    | 7     | 31  | 1900 | 8/1/1900  |
| SE23    | 7     | 31  | 1912 | 8/1/1912  |
| SE24    | 7     | 31  | 1913 | 8/1/1913  |
| SE25    | 2     | 14  | 1900 | 2/15/1900 |
| SE26    | 2     | 14  | 1912 | 2/15/1912 |
| SE27    | 2     | 14  | 1913 | 2/15/1913 |
| SE28    | 2     | 29  | 1900 | ERROR     |
| SE29    | 2     | 29  | 1912 | 3/1/1912  |
| SE30    | 2     | 29  | 1913 | ERROR     |
| SE31    | 2     | 30  | 1900 | ERROR     |
| SE132   | 2     | 30  | 1912 | ERROR     |
| SE33    | 2     | 30  | 1913 | ERROR     |
| SE34    | 2     | 31  | 1900 | ERROR     |
| SE35    | 2     | 31  | 1912 | ERROR     |
| SE36    | 2     | 31  | 1913 | ERROR     |



# Commission Problem

Weak Robust Test Cases (Valid + Invalid Inputs):

- These test cases ensure that the system handles both valid and invalid input classes.

Strong Robust Test Cases (Combinations of Invalid Inputs):

- These test cases test combinations of invalid inputs, ensuring the system's robustness in handling errors.

Output Range-Based Test Cases (Testing Sales and Commission Ranges):

- These test cases focus on ensuring the commission function outputs correct results for various sales ranges based on the values of locks, stocks, and barrels.

# Guidelines and Considerations

- The traditional form of equivalence testing is generally *not as thorough as* weak equivalence testing, and in its turn, *not as thorough as* strong equivalence testing
- If error conditions is a priority we can extend strong equivalence testing to include invalid classes
- Equivalence class testing is appropriate when input data is defined in terms of ranges and sets of discrete values.
- Logic of functionality of the program can help define the equivalence classes
- Strong equivalence takes the presumption that variables are independent, otherwise it generates some “error” test cases
- Can be strengthened by using it with domain testing (boundary value):
  - Reuse the work to define the ranges
  - Does not consider elements at equivalence class boundaries
  - Need to expand ECT to include BVT-like requirements (domain testing)

# **Decision Table-Based Testing**

# Decision Tables - General

- **Decision tables** have been used to represent and analyze complex logical relationships (since 1960s).
- Decision tables, like if-then-else and switch-case statements, associate conditions with actions to perform.
- A decision table has four portions: the left-most column is the stub portion; and to the right is the entry portion.
- The condition portion is noted by c's, and the action portion is noted by a's.
- Thus, decision table consisting of 4 areas called the *condition stub*, *condition entry*, *action stub* and the *action entry*

# Decision Tables - Structure

|                                        |                                                        |
|----------------------------------------|--------------------------------------------------------|
| Conditions - ( <i>Condition stub</i> ) | Condition Alternatives –<br>( <i>Condition Entry</i> ) |
| Actions – ( <i>Action Stub</i> )       | Action Entries                                         |

- Each condition corresponds to a variable, relation or predicate
- Possible values for conditions are listed among the condition alternatives
  - Boolean values (true/false, yes/no, 0/1) – Limited Entry Decision Tables
  - Several values – Extended Entry Decision Tables
  - Don't care value
- Each action is a procedure or operation to perform
- The entries specify whether (or in what order) the action is to be performed

- A column in the entry portion is a rule.
- Rules indicate which actions, if any, are taken for the circumstances indicated in the condition portion of the rule.

**Condition entry**

**Table 7.1   Portions of a Decision Table**

|                       |             |               |               |                   |               |               |                   |
|-----------------------|-------------|---------------|---------------|-------------------|---------------|---------------|-------------------|
| <b>Condition stub</b> | <i>Stub</i> | <i>Rule 1</i> | <i>Rule 2</i> | <i>Rules 3, 4</i> | <i>Rule 5</i> | <i>Rule 6</i> | <i>Rules 7, 8</i> |
|                       | c1          | T             | T             | T                 | F             | F             | F                 |
|                       | c2          | T             | T             | F                 | T             | T             | F                 |
|                       | c3          | T             | F             | —                 | T             | F             | —                 |
| <b>Action stub</b>    | a1          | X             | X             |                   | X             |               |                   |
|                       | a2          | X             |               |                   |               | X             |                   |
|                       | a3          |               | X             |                   | X             |               |                   |
|                       | a4          |               |               | X                 |               |               | X                 |

**Action Entry**

- The condition portion of a DT is a truth table that has been rotated 90°

# Decision Tables - Usage

- The use of the decision-table model is applicable when :
  - the specification is given or can be converted to a decision table .
  - once a rule is satisfied and the action selected, no other rule need be examined.
  - the order of executing actions in a satisfied rule is of no consequence.

# Decision Tables - Issues

Before using the tables, ensure:

- Rules must be complete:  
every combination of predicate truth values plus default cases are explicit in the decision table
- Rules must be consistent:  
every combination of predicate truth values results in only one action or set of actions



# Test Case Design

To identify test cases with decision tables, we interpret conditions as inputs, and actions as outputs.

Sometimes conditions end up referring to equivalence classes of inputs, and actions refer to major functional processing portions of the item being tested.

The rules are then interpreted as test cases.

One helpful style of producing DT is to add an action to show when a rule is logically impossible.

**Table 7.2 Decision Table for the Triangle Problem**

|                              |   |   |   |   |   |   |   |   |   |
|------------------------------|---|---|---|---|---|---|---|---|---|
| c3: a, b, c form a triangle? | N | Y | Y | Y | Y | Y | Y | Y | Y |
| c2: a = b?                   | — | Y | Y | Y | Y | N | N | N | N |
| c3: a = c?                   | — | Y | Y | N | N | Y | Y | N | N |
| c4: b = c?                   | — | Y | N | Y | N | Y | N | Y | N |
| a1: Not a triangle           | X |   |   |   |   |   |   |   |   |
| a2: Scalene                  |   |   |   |   |   |   |   |   | X |
| a3: Isosceles                |   |   |   |   | X |   | X | X |   |
| a4: Equilateral              |   | X |   |   |   |   |   |   |   |
| a5: Impossible               |   |   | X | X |   | X |   |   |   |

**Table 7.3 Refined Decision Table for the Triangle Problem**

|                    |   |   |   |   |   |   |   |   |   |   |   |
|--------------------|---|---|---|---|---|---|---|---|---|---|---|
| c1: $a < b + c$ ?  | F | T | T | T | T | T | T | T | T | T | T |
| c2: $b < a + c$ ?  | — | F | T | T | T | T | T | T | T | T | T |
| c3: $c < a + b$ ?  | — | — | F | T | T | T | T | T | T | T | T |
| c4: $a = b$ ?      | — | — | — | T | T | T | T | F | F | F | F |
| c5: $a = c$ ?      | — | — | — | T | T | F | F | T | T | F | F |
| c6: $b = c$ ?      | — | — | — | T | F | T | F | T | F | T | F |
| a1: Not a triangle | X | X | X |   |   |   |   |   |   |   |   |
| a2: Scalene        |   |   |   |   |   |   |   |   |   |   | X |
| a3: Isosceles      |   |   |   |   |   |   | X |   | X | X |   |
| a4: Equilateral    |   |   |   | X |   |   |   |   |   |   |   |
| a5: Impossible     |   |   |   |   | X | X |   | X |   |   |   |

# Decision table with rule count

| Stub              | R1 | R2 | R3 | R4 | R5 | R6 | R7 | R8 | R9 | R10 | R11 |
|-------------------|----|----|----|----|----|----|----|----|----|-----|-----|
| C1:a<b+c?         | F  | T  | T  | T  | T  | T  | T  | T  | T  | T   | T   |
| C2:b<a+c?         | -  | T  | T  | T  | T  | T  | T  | T  | T  | F   | T   |
| C3:c<a+b?         | -  | T  | T  | T  | T  | T  | T  | T  | T  | -   | F   |
| C4:a=b?           | -  | T  | T  | T  | T  | F  | F  | F  | F  | -   | -   |
| C5=a=c?           | -  | T  | T  | F  | F  | T  | T  | F  | F  | -   | -   |
| C6:b=c?           | -  | T  | F  | T  | F  | T  | F  | T  | F  | -   | -   |
| Rule count        | 32 | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 16  | 8   |
| A1:not a triangle | X  |    |    |    |    |    |    |    |    | X   | X   |
| A2:scalene        |    |    |    |    |    |    |    |    | X  |     |     |
| A3:isosceles      |    |    |    |    | X  |    | X  | X  |    |     |     |
| A4:equilateral    |    | X  |    |    |    |    |    |    |    |     |     |
| A5:impossible     |    |    | X  | X  |    | X  |    |    |    |     |     |

- Mutually exclusive conditions are situations where only one condition can be true at any time. For instance, if you have conditions like "Is it Day?" and "Is it Night?", both can't be true simultaneously.

# Decision table with mutually Exclusive conditions

| conditions      | R1 | R2 | R3 |
|-----------------|----|----|----|
| C1:month in M1? | T  | -  | -  |
| C2:month in M2? | -  | T  | -  |
| C3:month in M3? | -  | -  | T  |
| a1              |    |    |    |
| a2              |    |    |    |
| a3              |    |    |    |

# Rule count Decision table with mutually Exclusive conditions

| conditions      | R1 | R2 | R3 |
|-----------------|----|----|----|
| C1:month in M1? | T  | -  | -  |
| C2:month in M2? | -  | T  | -  |
| C3:month in M3? | -  | -  | T  |
| Rule count      | 4  | 4  | 4  |
| a1              |    |    |    |
| a2              |    |    |    |
| a3              |    |    |    |

# Test Cases for the Triangle Problem

| Case ID | a | b | c | Expected Output |
|---------|---|---|---|-----------------|
| DT1     | 4 | 1 | 2 | Not a Triangle  |
| DT2     | 1 | 4 | 2 | Not a Triangle  |
| DT3     | 1 | 2 | 4 | Not a Triangle  |
| DT4     | 5 | 5 | 5 | Equilateral     |
| DT5     | ? | ? | ? | Impossible      |
| DT6     | ? | ? | ? | Impossible      |
| DT7     | 2 | 2 | 3 | Isosceles       |
| DT8     | ? | ? | ? | Impossible      |
| DT9     | 2 | 3 | 2 | Isosceles       |
| DT10    | 3 | 2 | 2 | Isosceles       |
| DT11    | 3 | 4 | 5 | Scalene         |



# Decision Table for NextDate (First Attempt)

- Let us consider the following equivalence classes:

M1= {month | month has 30 days}

M2= {month | month has 31 days}

M3= {month | month is February}

D1= {day |  $1 \leq \text{day} \leq 28$ }

D2= {day | day = 29}

D3= {day | day = 30}

D4= {day | day=31}

Y1= {year | year = 1900}

Y2= {year |  $1812 \leq \text{year} \leq 2012$  AND  $\text{year} \neq 1900$  AND  $(0 = \text{year mod } 4)$ }

Y3= {year |  $1812 \leq \text{year} \leq 2012$  AND  $0 \neq \text{year mod } 4$ }

# Decision Table for NextDate (1)

| Conditions          | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |
|---------------------|----|----|----|----|----|----|----|----|
| C1: month in        | M1 | M1 | M1 | M1 | M2 | M2 | M2 | M2 |
| C2: day in          | D1 | D2 | D3 | D4 | D1 | D2 | D3 | D4 |
| C3: year in         | -  | -  | -  | -  | -  | -  | -  | -  |
| Rule count          | 3  | 3  | 3  | 3  | 3  | 3  | 3  | 3  |
| Actions             |    |    |    |    |    |    |    |    |
| A1: Impossible      |    |    |    | X  |    |    |    |    |
| A2: Increment day   | X  | X  |    |    | X  | X  | X  |    |
| A3: Reset day       |    |    | X  |    |    |    |    | X  |
| A4: Increment month |    |    | X  |    |    |    |    | ?  |
| A5: reset month     |    |    |    |    |    |    |    | ?  |
| A6: Increment year  |    |    |    |    |    |    |    | ?  |

# Decision Table for NextDate (2)

| Conditions          | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|---------------------|----|----|----|----|----|----|----|----|
| C1: month in        | M3 | M3 | M3 | M3 | M3 | M3 | M3 | M3 |
| C2: day in          | D1 | D1 | D1 | D2 | D2 | D2 | D3 | D3 |
| C3: year in         | Y1 | Y2 | Y3 | Y1 | Y2 | Y3 | -  | -  |
| Rule count          | 1  | 1  | 1  | 1  | 1  | 1  | 3  | 3  |
| Actions             |    |    |    |    |    |    |    |    |
| A1: Impossible      |    |    |    | X  |    | X  | X  | X  |
| A2: Increment day   |    | X  |    |    |    |    |    |    |
| A3: Reset day       | X  |    | X  |    | X  |    |    |    |
| A4: Increment month | X  |    | X  |    | X  |    |    |    |
| A5: reset month     |    |    |    |    |    |    |    |    |
| A6: Increment year  |    |    |    |    |    |    |    |    |

# Decision Table for NextDate (Third Attempt)

- Let us consider the following equivalence classes:

M1= {month | month has 30 days}

M2= {month | month has 31 days}

M3= {month | month is December}

M4= {month | month is February}

D1= {day |  $1 \leq \text{day} \leq 27$ }

D2= {day | day = 28}

D3= {day | day = 29}

D4= {day | day = 30}

D5= {day | day=31}

Y1= {year | year is a leap year}

Y2= {year | year is a common year}

# Decision Table for NextDate (1)

| Conditions          | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
|---------------------|----|----|----|----|----|----|----|----|----|----|
| C1: month in        | M1 | M1 | M1 | M1 | M1 | M2 | M2 | M2 | M2 | M2 |
| C2: day in          | D1 | D2 | D3 | D4 | D5 | D1 | D2 | D3 | D4 | D5 |
| C3: year in         | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  |
| Actions             |    |    |    |    |    |    |    |    |    |    |
| A1: Impossible      |    |    |    |    | X  |    |    |    |    |    |
| A2: Increment day   | X  | X  | X  |    |    | X  | X  | X  | X  |    |
| A3: Reset day       |    |    |    | X  |    |    |    |    |    | X  |
| A4: Increment month |    |    |    | X  |    |    |    |    |    | X  |
| A5: reset month     |    |    |    |    |    |    |    |    |    |    |
| A6: Increment year  |    |    |    |    |    |    |    |    |    |    |

# Decision Table for NextDate (2)

| Conditions          | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 |
|---------------------|----|----|----|----|----|----|----|----|----|----|----|----|
| C1: month in        | M3 | M3 | M3 | M3 | M3 | M4 | M4 | M4 | M4 | M4 | M4 | M4 |
| C2: day in          | D1 | D2 | D3 | D4 | D5 | D1 | D2 | D2 | D3 | D3 | D4 | D5 |
| C3: year in         | -  | -  | -  | -  | -  | -  | Y1 | Y2 | Y1 | Y2 | -  | -  |
| Actions             |    |    |    |    |    |    |    |    |    |    |    |    |
| A1: Impossible      |    |    |    |    |    |    |    |    |    | X  | X  | X  |
| A2: Increment day   | X  | X  | X  | X  |    | X  | X  |    |    |    |    |    |
| A3: Reset day       |    |    |    |    | X  |    |    | X  | X  |    |    |    |
| A4: Increment month |    |    |    |    |    |    |    | X  | X  |    |    |    |
| A5: reset month     |    |    |    |    | X  |    |    |    |    |    |    |    |
| A6: Increment year  |    |    |    |    | X  |    |    |    |    |    |    |    |

# Guidelines and Observations

- Decision Table testing is most appropriate for programs where
  - there is a lot of decision making
  - there are important logical relationships among input variables
  - There are calculations involving subsets of input variables
  - There are cause and effect relationships between input and output
  - There is complex computation logic (high cyclomatic complexity)
- Decision tables do not scale up very well
- Decision tables can be iteratively refined

# FAULT BASED TESTING

- Fault-based testing is a white-box testing method where test cases are designed based on the assumption that specific types of faults are likely to exist in the code.
- The goal is to prove that the software does not contain certain faults.



# Fault seeding

- Fault seeding (also known as error seeding or bug seeding) is a software testing technique where known faults (bugs) are intentionally inserted into a program to evaluate the effectiveness of the testing process.

# Assumptions in Fault based testing

- Effectiveness of fault based testing depends on..
  - The quality of the fault model and
  - On some basic assumptions about the relation of the seeded faults to faults that might actually be present.
- Seeded faults are small syntactic changes
- For example..
  - Replacing one variable reference by another in an expression
  - Changing a comparison from  $<$  to  $<=$

# Competent programmer hypothesis

- The Competent Programmer Hypothesis (CPH) assumes that a competent programmer writes code that is close to being correct — meaning, if there are faults, they are small, simple, and local, not massive or entirely incorrect implementations.

# Coupling effect hypothesis

- The Coupling Effect Hypothesis states that test cases that detect simple faults are also effective at detecting more complex (coupled) faults, which are combinations of those simple faults.

# Fault based testing: Terminology

## 1. Original program

The program unit (e.g., C function or java class) to be tested

## 2. Program location

- A region in the source code
- Typical locations are statements, arithmetic and Boolean expressions , and procedure calls

### 3. Alternate expression

- Source code text that can be legally substituted for the text at a program location
- A substitution is legal if the resulting program is syntactically correct(i.e, it compiles without errors)

### 4. Alternate program

- A program obtained from the original program by substituting an alternate expression for the text at some program location

5. Distinct behavior of an alternate program  $R$  for a test  $t$ 
  - The behavior of an alternate program  $R$  is distinct from the behavior of the original program  $P$  for a test  $t$ , if  $R$  and  $P$  produce a different result for  $t$ , or if the output of  $R$  is not defined for  $t$
  
6. Distinguished set of alternate programs for a test suite  $T$ 
  - A set of alternate programs are distinct if each alternate program in the set can be distinguished from the original program by at least one test in  $T$

# Mutation analysis

- It is the most common form of software fault-based testing.
- Mutation analysis involves modifying a program's source code in small ways to create mutants, and then running the test suite to see if the tests can detect the changes.



# Mutation analysis: Terminology

- Original program under test
  - The program or procedure(function) to be tested
- Mutant
  - A program that differs from the original program for one syntactic element
  - (e.g., a statement, a condition, a variable, a label)

- Distinguished mutant
  - A mutant that is detected by at least one test case in the test suite because its behavior differs from the original program.
- Equivalent mutant
  - A mutant that cannot be distinguished from the original program.
- Mutation operator
  - A rule for producing a mutant program by syntactically modifying the original program.

- Valid mutant
  - A mutant is valid if it is syntactically correct
- Useful mutant
  - A mutant is useful if, in addition to being valid, its behavior differs from the behavior of the original program for a small subset of test cases.

# Variations on mutation analysis

- In Mutation analysis process, mutants are killed based on the outputs produced by execution of test cases, it is known as **strong mutation**.
- Each mutant must be compiled and executed with each test case until it is killed.
- The time and space required for compiling all mutants and for executing all test cases for each mutant may be impractical.

- The computational effort required for mutation analysis can be reduced by decreasing the no. of mutants generated and the no. of test cases to be executed.
- **Weak mutation analysis** decreases the no. of tests to be executed by killing mutants when they produce a different intermediate state, rather than waiting for a difference in the final result or observable program behaviour.

- A single program can be seeded with many faults with weak mutation.
- A “meta mutant” program is divided into segments containing original as well as mutated source code, with a mechanism to select which segments to execute.
- Mutation analysis can be used either to judge the thoroughness of a test suite or to guide selection of additional test cases

- Statistical mutation analysis

Statistical sampling may keep the sample small enough to permit careful examination of equivalent mutants.

- The limitation is that partial coverage is meaningful only to the extent that the generated mutants are a valid statistical model of occurrence frequencies of actual faults
- Fault seeding can be used statistically in another way: To estimate the no. of faults remaining in a program

# Fault based adequacy criteria`

Given a program and a test suite T, mutation analysis consists of the following steps:

1. Select mutation operators  
Specific classes of faults
2. Generate mutants  
mutation operators to the original program
3. Distinguish mutants
  - execute original program and each generated mutant with the test cases in T
  - A mutant is **killed** when it can be distinguished from the original program



all the corresponding mutants. For example, a test case that fails the coverage adequacy criterion might not kill an sdl mutant if the value computed at the statement does not affect the behavior of the program on some possible executions.

| ID    | Operator | line | Original/<br>Mutant | 1U | 1D | 2U | 2D | 2M | End | Long | Mixed |
|-------|----------|------|---------------------|----|----|----|----|----|-----|------|-------|
| $M_i$ | ror      | 28   | (pos >= BUFLen-2)   | -  | -  | -  | -  | -  | -   | -    | -     |
|       |          |      | (pos == BUFLen-2)   | -  | -  | -  | -  | -  | -   | -    | -     |
| $M_j$ | ror      | 32   | (pos > 0)           | -  | x  | x  | x  | x  | -   | -    | -     |
|       |          |      | (pos >= 0)          | -  | -  | -  | -  | -  | -   | -    | -     |
| $M_k$ | sdl      | 16   | atCR = 0            | -  | -  | -  | -  | -  | -   | -    | -     |
|       |          |      | nothing             | -  | -  | -  | -  | -  | -   | -    | -     |
| $M_l$ | ssr      | 16   | atCR = 0            | -  | -  | -  | -  | -  | -   | -    | x     |
|       |          |      | pos = 0             | -  | -  | -  | -  | -  | -   | -    | x     |

| Test case | Description              | Test case | Description                                     |
|-----------|--------------------------|-----------|-------------------------------------------------|
| 1U        | One line, Unix line-end  | 2M        | Two lines, Mac line-end                         |
| 1D        | One line, DOS line-end   | End       | Last line not terminated with line-end sequence |
| 2U        | Two lines, Unix line-end | Long      | Very long line (greater than buffer length)     |
| 2D        | Two lines, DOS line-end  | Mixed     | Mix of DOS and Unix line ends in the same file  |

Figure 16.3: A sample set of mutants for program Transduce generated with mutation operators from Figure 16.2. x indicates the mutant is killed by the test case in the column head.

Test Suite  $TS = \{1U, 1D, 2U, 2D, 2M, \text{End}, \text{Long}\}$

- Kills  $M_j$
- $M_i$ ,  $M_k$  and  $M_l$  are not distinguished from the original program by any test in  $TS$
- Mutants not killed by a test suite are **live**
- A mutant can remain live for 2 reasons:
  - The mutant can be distinguished from the original program, but the test suite  $T$  does not contain a test case that distinguishes them
  - The mutant cannot be distinguished from the original program by any test case

- Given a set of mutants  $SM$  and a test suite  $T$
- The fraction of non equivalent mutants killed by  $T$  measures the adequacy of  $T$  with respect to  $SM$